

Motor de cálculo del Asistente de IA para Física I

Documento para colaboradores

Este documento es para los compañeros que van a construir las herramientas de cálculo del asistente de IA de Física I. Su foco principal es el **motor**: el **compilador**, el **solver** y el **validador**, escritos en Python. También explica las otras piezas del sistema para que todo encaje al integrarlo.

El objetivo es que entiendan **qué están construyendo, para qué sirve dentro del sistema completo, cómo se trabaja en el repositorio y qué reglas tienen que respetar sí o sí** para que las piezas hechas por grupos separados encajen entre sí.

1. ¿Qué es este proyecto?

Estamos construyendo un asistente de IA para la materia Física I de la universidad, que:

- Responde preguntas de los alumnos usando **solo** el contenido de los libros y guías oficiales de la cátedra (nunca inventa información de otras fuentes).
- Puede **graficar** situaciones físicas (movimientos, fuerzas, energía, oscilaciones) para ayudar a entender mejor los conceptos.
- Va a estar disponible en una interfaz web, pensado para que lo usen cientos de alumnos al mismo tiempo, especialmente en épocas de parciales.

Ustedes van a construir las piezas que le dan al asistente la capacidad de **calcular física real** antes de graficar, que es la parte más delicada de todo el sistema. Los errores/bugs en esta sección implican un error de cálculo mostrado como si fuera correcto a un alumno que está aprendiendo en la materia.

2. Qué se construye y por qué hay tres tipos de herramientas

Analizando las guías reales de la cátedra descubrimos que **no todos los problemas de física se resuelven de la misma manera** dentro del sistema. Hay tres tipos de situaciones, y cada una necesita una herramienta distinta:

- **Grupo A — Funciones fijas**: situaciones con fórmula ya conocida y cerrada.
- **Grupo B — Resolución numérica (ODE)**: fuerzas que dependen de la posición o la velocidad del cuerpo, sin una fórmula de libro que las resuelva directo.
- **Grupo C — Compilador + Solver + Validador (el motor)**: sistemas con varios cuerpos o elementos que se pueden combinar de formas que no se pueden anticipar todas de antemano.

Estas partes son piezas complementarias del mismo sistema. El asistente usa la pieza que corresponda según el tipo de consulta del alumno.

Este documento **no dice qué temas o funciones puntuales le corresponden a cada grupo**. Eso lo definen ustedes, junto con los ayudantes de cátedra, analizando el programa de la materia y decidiendo para cada tipo de ejercicio cuál de las tres categorías lo resuelve mejor. Lo que sí les damos acá es el **criterio para decidir** y ejemplos concretos de cada categoría.

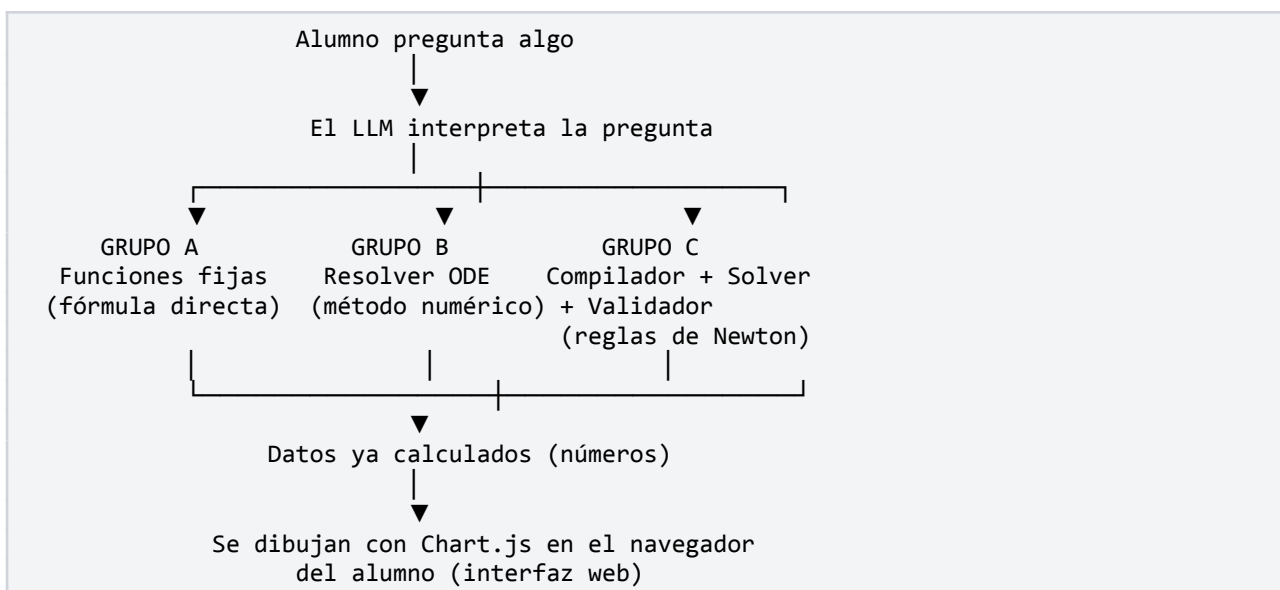
Criterio para decidir la categoría de cada situación

Háganse estas preguntas, en este orden, para cada tipo de ejercicio:

1. **¿Existe una fórmula cerrada y conocida que lo resuelve directamente, sin importar qué números le pongan?** Si sí, es candidato a **función fija**.
 - La posición en caída libre siempre es $x(t) = x_0 + v_0*t + 0.5*a*t^2$. No importa qué valores de x_0 , v_0 , a les den: la fórmula no cambia.
 - La velocidad tangencial en movimiento circular uniforme siempre es $v = \omega*r$.
2. **¿Hay una fuerza que depende de dónde está el cuerpo o de qué tan rápido va, y eso hace que la ecuación no tenga una solución simple de “despejar a mano”?** Si sí, es candidato a **resolución numérica (ODE)**.
 - Un paracaidista con resistencia del aire proporcional al cuadrado de la velocidad ($F = -k*v^2$): la aceleración depende de la propia velocidad y no siempre se puede despejar $x(t)$ con una fórmula de libro.
 - Un auto que frena solo por rozamiento viscoso ($F = -k*v$): la fuerza depende de la velocidad en cada instante.
3. **¿Hay varios cuerpos, cuerdas, poleas, resortes o superficies que se pueden combinar de muchas formas, al punto de que sería imposible escribir una función para cada combinación?** Si sí, es candidato a **compilador + solver**.
 - Un bloque sobre un plano inclinado, conectado por una cuerda que pasa por una polea a otro bloque que cuelga. Mañana podría ser el mismo sistema con un resorte en el medio, con un tercer bloque o sin plano inclinado. Cada combinación nueva no debería requerir una función nueva desde cero.
 - Un cuerpo sobre una superficie con rozamiento, donde hay que decidir si desliza o no (depende de los números del problema, no se sabe de antemano).

Si no queda clara la categoría, o parece que podría entrar en más de una, es una buena señal para llevar el ejercicio a los ayudantes de cátedra y discutirlo en conjunto.

3. Cómo encaja todo (arquitectura general)



Ninguno de los grupos dibuja el gráfico directamente. Todos entregan **los datos ya calculados**, en un mismo formato (ver sección 5.2), y es el frontend (con Chart.js) el que los convierte en un gráfico visual. Están construyendo calculadoras de física confiables, no artistas de gráficos.

El motor (Grupo C) es un paquete Python aislado: **no usa IA ni red**. Todo es determinista y testeable.

4. Glosario mínimo

- **Escena:** descripción estructurada de qué hay en el problema (cuerpos, superficies, conexiones, incógnitas).
- **Compilador:** convierte la escena en ecuaciones aplicando reglas físicas fijas. Es la parte del *planteo*.
- **Solver:** resuelve el sistema de ecuaciones ya planteado (usa SymPy/SciPy). No necesita saber física.
- **Validador:** juzga si un resultado es físicamente creíble. No modifica el planteo.
- **Caso de oro:** par escena → resultado esperado, guardado como YAML.

Más detalle en [docs/glosario.md](#) del repositorio.

5. Contratos universales: aplican a todos los grupos, sin excepción

Estas reglas hacen que las piezas, construidas por grupos separados, encajen al integrarlas. **No son sugerencias, son requisitos de entrega.**

5.1 Firma de función: type hints + docstring obligatorios

El sistema (LlamaIndex) le muestra al LLM el nombre de cada función, sus parámetros y para qué sirve, **leyendo automáticamente el código que ustedes escriban**. Si falta un type hint o el docstring es pobre, el LLM entiende mal la herramienta y se equivoca al invocarla. Formato obligatorio:

```
def graficar_movimiento_rectilineo(
    x0: float,
    v0: float,
    a: float,
    t_max: float
) -> dict:
    """
    Grafica posición, velocidad y aceleración en función del tiempo
    para movimiento rectilíneo (MRU si a=0, MRUV si a≠0).

    Args:
        x0: Posición inicial en metros.
        v0: Velocidad inicial en m/s.
        a: Aceleración en m/s². Usar 0 para MRU.
        t_max: Tiempo máximo a graficar, en segundos.
    """
```

5.2 Formato de salida: igual para todos

Toda función-herramienta que alimente un gráfico (las que el LLM invoca y cuyo resultado llega al frontend) devuelve un diccionario con esta estructura, para que el frontend pueda dibujar cualquier resultado de la misma forma:

```
{
  "tipo_grafico": "linea",
  "ejes": {
    "x": {"label": "t (s)", "valores": [0, 0.1, 0.2, "..."]},
    "y": {"label": "x (m)", "valores": [0, 1.2, 2.4, "..."]}
  },
  "series": [
    {"nombre": "posición", "valores": [0, 1.2, 2.4, "..."]}
  ],
}
```

```
"metadata": {"unidad_x": "s", "unidad_y": "m"}
}
```

tipo_grafico puede ser "línea", "dispersion" o "vectores" (para diagramas de cuerpo libre). Si necesitan un tipo distinto, consulten con el equipo central antes de inventar un formato nuevo.

En el motor hay dos capas, y no se contradicen. Internamente, `resolver()` devuelve un modelo Pydantic Resultado (ver 7.5), que es el contrato entre módulos y entre repositorios. La función-herramienta que ve el LLM convierte ese Resultado al diccionario de arriba mediante un adaptador, `resultado_a_grafico(resultado: Resultado) -> dict`. El adaptador, la forma exacta del Resultado y la correspondencia entre ambos los fija el equipo central en `contrato/CONTRATO.md`. Los módulos del motor (compilador, solver y validador) trabajan siempre con modelos Pydantic y nunca arman este diccionario. No la inventen.

5.3 Manejo de errores: excepciones propias, no errores genéricos de Python

Si los parámetros son inválidos (una masa negativa, un ángulo imposible), o si el cálculo no se puede completar (una ecuación no converge, una hipótesis de planteo, por ejemplo de rozamiento o de cuerda, no es consistente), **no dejen que Python tire un error genérico**. Lancen la excepción que define el equipo central (algo como `ParametroInvalidoError` o `CalculoNoConvergeError`, entregadas en la plantilla de código base; en el motor viven en `src/motor/errores.py`). Así el backend le muestra al alumno un mensaje claro en vez de un error críptico.

5.4 Convención de signos y unidades: fijada por el equipo central

Todas las funciones asumen **unidades del Sistema Internacional** (metros, segundos, kilogramos, radianes) salvo que el docstring diga lo contrario explícitamente. La convención de signos (por ejemplo, “positivo = hacia arriba o hacia la derecha del movimiento”) la define el equipo central antes de empezar y **no la inventan ustedes**: un signo invertido hace que un vector se dibuje apuntando al revés, un error silencioso y difícil de detectar a simple vista. Ver sección 12.

5.5 Funciones puras: sin estado oculto

Dada la misma entrada, cada función devuelve siempre exactamente el mismo resultado. No usen variables globales, no lean nada de fuera de sus parámetros, no dependan del historial de la conversación. El sistema va a cachear resultados repetidos para atender a muchos alumnos a la vez: si la función no es pura, el caché puede devolver resultados viejos o inconsistentes.

5.6 Nunca ejecutar código libre generado por el LLM

Esta es la regla de seguridad más importante del proyecto. El LLM puede mandar una **fórmula matemática como texto** (por ejemplo `"-k*v**2"`), pero su código **nunca** debe ejecutarla con `exec()` o `eval()` de Python puro. Las expresiones matemáticas se evalúan con una librería restringida que indica el equipo central (`sympy` o `asteval`), que solo entiende operaciones matemáticas y no puede ejecutar instrucciones de programación. Si tienen dudas sobre si algo cuenta como “ejecutar código libre”, pregunten antes de implementar.

En la práctica, quien recibe fórmulas como texto es el Grupo B (`fuerza_expresion`), y ahí aplica el parseo seguro (ver sección 6). El motor (Grupo C) recibe una escena estructurada, validada con Pydantic, y **no acepta fórmulas en texto libre**; tampoco usa `eval()` ni `exec()` en ninguno de sus módulos.

5.7 Testeo aislado: cada función se prueba sola

Escriban las funciones para que se puedan probar con `pytest` sin levantar todo el sistema (sin LLM, sin API, sin base de datos). Reciben datos simples de entrada y devuelven un resultado que se compara directo contra uno esperado. El equipo central entrega casos ya resueltos a mano para que puedan validar su implementación antes de entregarla (ver secciones 10 y 14).

5.8 Las funciones corren en un threadpool

Decisión de arquitectura ya resuelta por el equipo central: **no hace falta que su código sea asíncrono**. Escriban funciones normales (síncronas); el backend las invoca de forma que no bloqueen al resto del sistema. Es especialmente relevante para el Grupo B, cuyos cálculos pueden tardar más que una fórmula simple.

5.9 Qué NO tienen que construir (fuera de alcance)

- **Nadie** construye un sandbox de ejecución de código libre: fue evaluado y descartado por riesgo de seguridad.
- El alcance de temas de cada grupo es el que definan junto con los ayudantes de cátedra (ver sección 2). Una vez acordado y escrito, **no lo cambien por su cuenta sobre la marcha**. Si un grupo descubre que un tema asignado encaja mejor en otra categoría, se discute entre los grupos y con el equipo central antes de mover nada.

6. Las otras piezas del sistema (contexto): Grupo A y Grupo B

Si trabajan en el motor no tienen que construir estas piezas, pero conviene que sepan qué hacen y cuándo una situación les corresponde a ellas y no al motor.

Grupo A — Funciones fijas

Funciones que calculan y grafican situaciones con **fórmula ya conocida y cerrada**: el cálculo es directo y no depende de combinar elementos de formas distintas. Cada grupo arma, junto con los ayudantes de cátedra, su lista de situaciones, usando el punto 1 del criterio de la sección 2. Como guía, **no como lista cerrada**:

- Movimientos cuya ecuación de posición/velocidad/aceleración ya se sabe de antemano (el tipo de movimiento define la fórmula; los números del problema son solo parámetros).
- Magnitudes que se calculan con una sola fórmula directa (una relación, un teorema aplicado en un solo paso).
- Casos “ideales” de un fenómeno, sin fuerzas adicionales que compliquen la ecuación.

Calculan con una fórmula directa y arman el diccionario de salida en la misma función; no hace falta separar “cálculo” de “gráfico”. Cada función define nombre, parámetros con tipo y unidad, y qué devuelve (formato de 5.1).

Señal de alerta: si mientras definen una función se dan cuenta de que “depende de si tal otra cosa pasa” (hay o no rozamiento, el ángulo es chico o grande, hay más de un cuerpo), probablemente esa situación no es tan “cerrada” como parecía. Discutan con los otros grupos si encaja mejor como ODE o como compilador.

Grupo B — Resolución numérica (ODE)

Una herramienta que resuelve el movimiento de un cuerpo cuando la fuerza depende de su posición (x) y/o su velocidad (v) y no existe una fórmula cerrada simple. Ejemplos del *tipo* de situación: fuerzas resistivas que dependen de la velocidad (lineales o cuadráticas), osciladores con amortiguamiento, péndulos a ángulos grandes donde no vale la aproximación de ángulo pequeño. Identificar cuáles del programa real corresponden es trabajo del grupo, con los ayudantes de cátedra.

Contrato de la función:

```
def resolver_movimiento_numerico(
    masa: float,
    fuerza_expresion: str,
    x0: float,
    v0: float,
    t_max: float,
```

```

    metodo: str = "RK45"
) -> dict:
    """
    Resuelve numéricamente el movimiento de un cuerpo bajo una fuerza
    que puede depender de la posición (x) y/o la velocidad (v).

    Args:
        masa: masa del cuerpo en kg.
        fuerza_expresion: expresión matemática de  $F(x, v)$  como texto,
            usando solo las variables  $x$ ,  $v$  y constantes numéricas.
            Ejemplos:  $-k*v**2$ ,  $-b*v - k*x$ .
        x0: posición inicial en metros.
        v0: velocidad inicial en m/s.
        t_max: tiempo máximo a simular, en segundos.
        metodo: método numérico de integración (ver scipy.integrate.solve_ivp).
    """

```

Decisiones técnicas ya tomadas:

- Usar `scipy.integrate.solve_ivp`, con RK45 por defecto. No inventen un método de integración propio.
- Primero intentar `sympy.dsolve` (solución exacta); si no hay una solución simbólica **explícita**, recién ahí `scipy`. Ojo: `dsolve` no siempre lanza `NotImplementedError` cuando no puede resolver. Puede devolver una integral sin evaluar (`Integral`), una solución implícita o una lista de ramas. Por eso hay que comprobar el resultado antes de aceptarlo:

```

def solucion_exacta_o_none(ecuacion, x_t, condiciones_iniciales):
    """Devuelve x(t) explícita, o None si hay que integrar numéricamente."""
    try:
        sol = sympy.dsolve(ecuacion, x_t, ics=condiciones_iniciales)
    except (NotImplementedError, ValueError, TypeError):
        return None
    if isinstance(sol, list):      # varias ramas: no hay solución única
        return None
    if sol.has(sympy.Integral):    # integral sin evaluar: no es solución cerrada
        return None
    if sol.lhs != x_t:            # solución implícita, no despejada
        return None
    return sol.rhs

x_exacta = solucion_exacta_o_none(ecuacion, x_t, condiciones_iniciales)
if x_exacta is None:
    resultado = scipy.integrate.solve_ivp(ecuacion_numerica, ...)

```

Lo que tiene que resolver el grupo:

- **Parseo seguro de fuerza_expresion** con la librería indicada por el equipo central, sin ejecutar código libre.
- **Detección de no convergencia**: si `solve_ivp` indica que falló, devolver el error correspondiente (5.3), no un gráfico con datos sin sentido.
- **Validación básica de la expresión** antes de integrar: ¿usa solo x , v , t y números? Si no, error antes de llegar al solver.

7. Grupo C — Compilador, solver y validador

Qué se construye

Para situaciones con varios cuerpos, cuerdas, poleas, resortes y superficies que se combinan de muchas formas, escribir una función por combinación es inviable. En cambio se construye un sistema que:

1. Recibe una **escena** (descripción estructurada de la situación física).

2. Aplica reglas de la física de Newton para armar las ecuaciones (el **compilador**).
3. Resuelve esas ecuaciones con SymPy (el **solver**).
4. Verifica que el resultado sea físicamente coherente antes de graficar (el **validador**).

7.1 La escena (entrada del sistema)

Es un JSON con vocabulario físico general, no una situación específica:

```
{
  "cuerpos": [{"id": "A", "masa": {"valor": 5, "unidad": "kg"}}],
  "superficies": [{"id": "plano", "angulo": {"valor": 30, "unidad": "deg"},
    "mu_estatico": 0.3, "mu_cinetico": 0.2}],
  "contactos": [{"cuerpo": "A", "superficie": "plano"}],
  "conexiones": [],
  "condiciones_iniciales": {"A": {"v0": 0}},
  "incognitas": ["aceleracion_A"]
}
```

La idea central: con un vocabulario de 10-15 elementos genéricos (cuerpo, superficie, conexión por cuerda, polea, resorte) se puede describir cualquier combinación de las situaciones que el grupo decida cubrir, sin una función nueva por cada situación. La escena es un dato estructurado validado por los modelos Pydantic: no contiene fórmulas en texto ni se evalúa como código. El esquema formal (versionado) vive en `contrato/esquema_escena.schema.json`, y los modelos Pydantic en `src/motor/esquema/`.

7.2 El compilador (reglas de Newton, sin IA de por medio)

Es código determinista: reglas fijas, no un modelo de lenguaje. En el repo hay **una regla por archivo** (`src/motor/compilador/`). Pasos típicos:

1. **Generar el diagrama de cuerpo libre automáticamente:** para cada cuerpo, identificar qué fuerzas actúan según lo que tenga (peso siempre, normal si hay contacto, rozamiento si la superficie tiene μ , tensión si hay cuerda, fuerza elástica si hay resorte).
2. **Elegir ejes de proyección** (por ejemplo, paralelo y perpendicular al plano).
3. **Proyectar las fuerzas** sobre esos ejes.
4. **Escribir la segunda ley de Newton** por cuerpo y por eje ($\Sigma F = m \cdot a$).
5. **Agregar restricciones:** cuerda inextensible $\rightarrow$ misma aceleración en los cuerpos conectados (solo mientras la cuerda esté tensa, es decir, $T \geq 0$); polea ideal $\rightarrow$ misma tensión en ambos lados.
6. **Manejar las hipótesis de planteo con verificación:** plantear una hipótesis, resolver, verificar que sea consistente y, si no, probar la alternativa. Hay dos familias, cada una con su causa física: el **rozamiento** (el cuerpo desliza, o no desliza), que verifica el propio compilador, y la **cuerda** (tensa o floja), que se cuestiona cuando el validador detecta una tensión negativa (ver 7.4).

Cada regla nueva se documenta en `docs/reglas-compilador.md` (la lista de reglas físicas y su estado).

7.3 El solver

Una vez planteado el sistema de ecuaciones, se resuelve con `sympy.solve()` (sistemas algebraicos) o `sympy.dsolve()` (si aparece una ecuación diferencial con solución exacta). Vive en `src/motor/solver/` como envoltorio de SymPy/SciPy. **El solver no necesita saber nada de física:** solo resuelve lo que el compilador le entrega ya planteado. Recibe ecuaciones simbólicas construidas por el compilador a partir de la escena estructurada, nunca texto libre.

`dsolve` no siempre lanza una excepción cuando no puede resolver (puede devolver una integral sin evaluar, una solución implícita o una lista de ramas). Por eso el solver comprueba el resultado, con el mismo criterio de la sección 6, y solo lo acepta si es una solución explícita. Si no lo es, lo trata como un caso sin solución y lanza `CalculoNoConvergeError`.

7.4 El validador de coherencia (antes de graficar)

Chequeos obligatorios después de resolver y antes de devolver el resultado. Cada validador es un archivo en `src/motor/validadores/`, con su severidad declarada, su regla física y su acción ante un fallo documentadas en `docs/validadores.md`:

```
def validar_tension_no_negativa(tension_calculada: float) -> bool:
    """Una cuerda no puede 'empujar' - si  $T < 0$ , el planteo es inconsistente."""
    return tension_calculada >= 0

def validar_suma_fuerzas_igual_ma(fuerzas: list, masa: float, aceleracion: float,
                                  tolerancia: float = 0.05) -> bool:
    """La suma vectorial de las fuerzas debe coincidir con  $m \cdot a$ ."""
    ...

def validar_normal_perpendicular(vector_normal, angulo_superficie: float,
                                  tolerancia: float = 0.01) -> bool:
    """La normal debe ser perpendicular a la superficie de contacto."""
    ...
```

El validador solo **juzga**; no modifica el planteo. Cada validador declara en `docs/validadores.md` qué causa física cuestiona cuando falla. Si un chequeo falla, el sistema **no muestra el gráfico**, y el motor (no el validador) actúa según esa declaración:

- **Tensión negativa** (`validar_tension_no_negativa`): una cuerda no puede empujar, así que la hipótesis “cuerda tensa” es la que no se sostiene. El compilador replantea con la **cuerda floja** (se elimina la restricción de cuerda inextensible, $T = 0$ y los cuerpos dejan de estar ligados), y se vuelve a resolver y validar. Cambiar la hipótesis de rozamiento no arregla este caso.
- **Suma de fuerzas distinta de $m \cdot a$ y normal no perpendicular**: indican un error de planteo o de resolución, no una hipótesis alternativa. No se reintentan: se devuelve el error.
- **Rozamiento (desliza o no desliza)**: lo verifica el compilador con su propia hipótesis y verificación (7.2, paso 6), no un validador posterior.

Los reintentos son acotados: a lo sumo uno por cada hipótesis alternativa. Si el resultado replanteado sigue sin validar, el motor devuelve el error correspondiente (5.3) para que el asistente le explique al alumno en texto que no pudo resolver ese caso, en vez de arriesgarse a mostrar un diagrama con un error de física silencioso.

7.5 Una única puerta de entrada: `resolver()`

El motor expone una sola función de cálculo pública, por ejemplo `resolver(escena: Escena) -> Resultado`, y todo lo demás es interno. `Resultado` es un modelo Pydantic, no el diccionario de 5.2: la conversión al formato de gráfico la hace el adaptador `resultado_a_grafico()`, que fija el equipo central (ver 5.2). El backend se integra contra esa función y ese adaptador, y nada más. Su firma, el formato del resultado y el de los errores los fija el equipo central en `contrato/CONTRATO.md` antes de que empiece el trabajo. Los modelos Pydantic de cada frontera (`Escena`, `SistemaEcuaciones`, `Solucion`, `ResultadoValidacion`, `Error`) permiten que cada persona pruebe su módulo con entradas fabricadas, sin depender del módulo de otro. El esquema se exporta a JSON Schema y se reutiliza para estructurar la salida del LLM y para el frontend.

7.6 Qué queda fuera del alcance

1. No construyen un sandbox de ejecución libre como respaldo. Si la escena no encaja en el esquema, o el compilador no tiene reglas para ese caso, el sistema explica en texto y no grafica, punto.
2. El alcance de temas del compilador (qué situaciones entran y cuáles no) lo definen ustedes con los ayudantes de cátedra, usando el criterio de la sección 2. Una vez acordado y documentado, no lo amplíen por su cuenta sin avisar al resto de los grupos ni al equipo central.

8. El repositorio del motor

El motor se desarrolla en un **repositorio aparte** (fisica-motor). Ustedes implementan el motor de forma independiente y el equipo central lo integra después con el backend. Así tienen una superficie chica de trabajo y no pueden romper el resto del sistema. Backend, frontend e infraestructura no existen en este repo.

```
fisica-motor/
├── README.md
├── CONTRIBUTING.md
├── CHANGELOG.md
├── Makefile                # comandos make (ver README)
├── pyproject.toml          # versiones fijas
├── uv.lock
├── Dockerfile
├── .github/
│   ├── workflows/ci.yml    # tests + lint en cada PR
│   ├── CODEOWNERS          # quién revisa qué carpeta
│   └── pull_request_template.md
├── contrato/              # SOLO el equipo central lo modifica
│   ├── esquema_escena.schema.json
│   ├── CONTRATO.md         # firma de resolver(), formato de errores
│   └── VERSION
├── docs/
│   ├── glosario.md         # solver, compilador, validador, escena
│   ├── convenciones-fisicas.md # signos, ejes, g = 9,8, notación de la cátedra
│   ├── reglas-compilador.md # lista de reglas físicas y su estado
│   ├── validadores.md      # cada validador, severidad, regla física
│   └── decisiones/         # ADR: una página corta por decisión de diseño
├── src/motor/
│   ├── __init__.py         # expone resolver() y resultado_a_grafico()
│   ├── esquema/            # modelos Pydantic de la escena
│   ├── compilador/         # escena → ecuaciones (una regla por archivo)
│   ├── solver/             # ecuaciones → resultado (SymPy/SciPy)
│   ├── validadores/        # uno por archivo, con severidad declarada
│   └── errores.py          # categorías de error tipadas
├── tests/
│   ├── unitarios/
│   ├── propiedades/        # Hypothesis
│   ├── adversos/           # entradas que deben fallar de forma controlada
│   ├── regresion/          # un caso por cada bug encontrado
│   └── aceptacion/         # SOLO el equipo central (CODEOWNERS)
├── casos_oro/              # datos, no código
│   ├── README.md           # formato y cómo agregar casos
│   ├── schema_caso.json    # esquema del caso de oro
│   ├── dinamica/
│   ├── cinematica/
│   └── revisados/          # los que ya pasaron revisión cruzada
├── herramientas/
└── validar_casos.py        # chequea formato de los YAML antes de correr tests
```

README de arranque

El criterio de éxito del onboarding es este: clonás, corrés `make setup` && `make test` y ves los tests en verde **sin preguntarle a nadie**. Este es el contenido del README.md del repo:

Motor de cálculo - Física I

Paquete Python que recibe la descripción de un problema (la "escena"), plantea las ecuaciones con reglas físicas fijas, las resuelve y valida el resultado. No usa IA ni red. Todo es determinista y testeable.

Glosario mínimo

- ****Escena:**** descripción estructurada de qué hay en el problema (cuerpos, superficies, conexiones, incógnitas).
- ****Compilador:**** convierte la escena en ecuaciones aplicando reglas físicas. Es parte del planteo.
- ****Solver:**** resuelve el sistema de ecuaciones ya planteado (usa SymPy/SciPy).
- ****Validador:**** juzga si un resultado es físicamente creíble. No modifica el planteo.
- ****Caso de oro:**** par escena → resultado esperado, guardado como YAML.

Más detalle en ``docs/glosario.md``.

Arranque en 10 minutos

Requisitos: Docker (o Python 3.12 y ``uv``).

```
```bash
git clone <URL-del-repo>
cd fisica-motor
make setup # instala dependencias
make test # debería terminar en verde
```
```

Si ``make test`` no da verde en tu máquina, es un problema del repo: abrí un issue.

Comandos

| Comando | Qué hace |
|-----------------------------------|---|
| <code>`make test-fast`</code> | Tests rápidos (unitarios y casos de oro). Correr antes de cada commit |
| <code>`make test`</code> | Todo, incluidos tests de propiedades |
| <code>`make lint`</code> | Formato y tipos (ruff, mypy) |
| <code>`make cov`</code> | Cobertura de tests |
| <code>`make validar-casos`</code> | Chequea el formato de los YAML en <code>`casos_oro/`</code> |

Cómo se trabaja

1. Tomá un issue asignado. Los criterios de aceptación están en el issue.
2. Creá una rama: ``modulo/descripcion-corta``.
3. Escribí primero el caso de oro y el test, después el código.
4. Abrí un Pull Request chico (una regla o un validador por PR).
5. Necesitás CI en verde y una aprobación de otra persona.

Qué NO podés modificar

Todo lo que está en ``contrato/``, ``tests/aceptacion/``, ``src/motor/__init__.py`` y ``src/motor/errores.py`` lo gestiona el equipo núcleo. Si creés que algo ahí debe cambiar, abrí un issue con la etiqueta ``contrato``.

Reglas importantes

- Lean ``docs/convenciones-fisicas.md`` antes de escribir nada (ejes, signos, g, unidades).
- Unidades internas en SI. Ángulos en radianes dentro del código.
- Nunca copies texto de libros: parafraseá los enunciados.
- Nunca subas claves, datos de alumnos ni PDFs.
- Quien implementa una regla no escribe sus propios casos de oro de revisión.

¿Dudas?

Preguntá en el canal del grupo o mencioná a @responsable-del-motor en el issue.

9. Cómo se trabaja

- Los **issues** los crea y asigna el equipo central, con criterios de aceptación escritos como condiciones verificables (“el caso plano_inclinado_03 pasa”, “la propiedad X se cumple en 500 escenas aleatorias”), no como “implementar rozamiento”.
- Trabajan en una **rama** (modulo/descripcion-corta), escriben **primero el caso de oro y el test**, luego el código.
- Abren un **Pull Request**. El CI debe estar en verde.

- Lo revisa **otra persona** (de preferencia no quien escribió los casos) y, si toca el contrato, alguien del equipo central.
- Se hace merge, y el caso pasa a `casos_oro/revisados/` cuando alguien distinto lo confirmó.

Mantengan los PR chicos: una regla o un validador por PR. Los PR grandes se revisan mal, y es donde se cueban los errores de signo.

Qué no pueden modificar: `contrato/`, `tests/acceptacion/`, `src/motor/__init__.py` y `src/motor/errores.py` son del equipo central. Si creen que algo ahí debe cambiar, abren un issue con la etiqueta `contrato`. Ustedes tocan reglas, validadores, solver y casos, pero no los contratos.

Etiquetas: hay etiquetas por módulo y por dificultad, y una etiqueta `buen-primer-issue` para empezar.

10. Casos de oro

Un caso de oro es un par escena → resultado esperado, guardado como **datos** (YAML), con tolerancia, fuente, tipo, dificultad y un campo `estado: borrador | revisado`. Un script valida el formato antes de correr los tests, así un YAML mal escrito falla con un mensaje claro y no con un error críptico en `pytest`.

Ejemplo (`casos_oro/dinamica/plano_inclinado_01.yaml`)

```
id: dinamica-plano-inclinado-01
estado: borrador          # borrador | revisado
tipo: plano_inclinado
dificultad: 1             # 1 (básico) a 5 (difícil)
enunciado: >
  Un bloque de 5 kg se suelta en reposo sobre un plano inclinado de 30°
  con coeficiente de rozamiento cinético 0,2. Hallar su aceleración.
escena:
  cuerpos:
    - {id: A, masa: {valor: 5, unidad: kg}}
  superficies:
    - {id: plano, angulo: {valor: 30, unidad: deg}, mu_cinetico: 0.2}
  contactos:
    - {cuerpo: A, superficie: plano}
  conexiones: []
  condiciones_iniciales:
    A: {v0: 0}
  incognitas: [aceleracion_A]
esperado:
  aceleracion_A: {valor: 3.20, unidad: "m/s^2"}
tolerancia: {relativa: 0.01}
oraculo: formula_cerrada  # formula_cerrada | resuelto_catedra | calculo_docente |
                          propiedad
fuente: "Guía 3, ej. 4 (parafraseado)"
autor: "@usuario"
revisado_por: null
notas: "a = g (sin θ - μ cos θ), con g = 9,8 m/s²"
```

Esquema (`casos_oro/schema_caso.json`)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Caso de oro",
  "type": "object",
  "required": ["id", "estado", "tipo", "dificultad", "enunciado", "escena",
    "esperado", "tolerancia", "oraculo", "fuente", "autor"],
  "additionalProperties": false,
  "properties": {
    "id": {"type": "string", "pattern": "^[a-z0-9]+(-[a-z0-9_]+)*$"},
```

```

"estado": {"enum": ["borrador", "revisado"]},
"tipo": {"type": "string"},
"dificultad": {"type": "integer", "minimum": 1, "maximum": 5},
"enunciado": {"type": "string", "minLength": 10},
"escena": {
  "type": "object",
  "description": "Debe validar contra contrato/esquema_escena.schema.json"
},
"esperado": {
  "type": "object",
  "minProperties": 1,
  "additionalProperties": {
    "type": "object",
    "required": ["valor", "unidad"],
    "properties": {
      "valor": {"type": "number"},
      "unidad": {"type": "string"}
    }
  }
},
"tolerancia": {
  "type": "object",
  "minProperties": 1,
  "properties": {
    "relativa": {"type": "number", "exclusiveMinimum": 0},
    "absoluta": {"type": "number", "exclusiveMinimum": 0}
  }
},
"oraculo": {"enum": ["formula_cerrada", "resuelto_catedra",
  "calculo_docente", "propiedad"]},
"fuentes": {"type": "string", "minLength": 3},
"autor": {"type": "string"},
"revisado_por": {"type": ["string", "null"]},
"notas": {"type": "string"}
},
"allOf": [
  {
    "if": {"properties": {"estado": {"const": "revisado"}}},
    "then": {"required": ["revisado_por"],
      "properties": {"revisado_por": {"type": "string"}}}
  }
]
}

```

Reglas que hace cumplir [herramientas/validar_casos.py](#)

El esquema solo no las expresa bien, así que las verifica el script:

- `revisado_por` debe ser **distinto de** `autor`.
- La `escena` debe validar contra el esquema de escena vigente en `contrato/`.

Cómo se usa el campo `oraculo`

Indica de dónde viene la verdad del caso (fórmula cerrada, resuelto por la cátedra, cálculo de un docente o propiedad). Permite reportar cuántos casos tienen verdad analítica, cuántos vienen de la cátedra y cuántos de un docente, lo que da peso al golden set en la defensa.

Aviso legal

En el campo `fuentes` pongan la referencia ("Guía 3, ej. 4"), pero **parafraseen los enunciados** y no copien texto de libros con derechos de autor al repositorio.

11. Automatización y plantillas

Automatización que hace cumplir las reglas

- **Ramas protegidas:** nada entra a `main` sin Pull Request, CI en verde y al menos una aprobación.
- **CODEOWNERS:** exige aprobación del equipo central en `contrato/`, `tests/aceptacion/`, `src/motor/__init__.py` y `src/motor/errores.py`.
- **Pre-commit:** formateo y lint (por ejemplo `ruff`) y chequeo de tipos (`mypy`) automáticos, para no discutir estilo en las revisiones.
- **CI en dos niveles:** un conjunto rápido en cada commit (segundos) y uno largo (propiedades pesadas, mutación con `mutmut`) nocturno o antes de cada hito.
- **Entorno reproducible:** dependencias con archivo de bloqueo (`uv.lock`) y la misma imagen Docker en local, CI y servidor.

Plantilla de Pull Request ([.github/pull_request_template.md](https://github.com/pull_request_template.md))

```
## Qué cambia
<!-- Una o dos frases. Un PR = una regla o un validador. -->

## Issue relacionado
Closes #

## Tipo
- [ ] Regla del compilador
- [ ] Validador
- [ ] Solver
- [ ] Caso(s) de oro
- [ ] Corrección de bug
- [ ] Documentación

## Checklist
- [ ] Agregué al menos un caso de oro (`casos_oro/`)
- [ ] Agregué o actualicé tests (unitario y, si aplica, de propiedad)
- [ ] Si arreglé un bug, agregué su caso en `tests/regresion/`
- [ ] `make test` pasa en mi máquina
- [ ] `make validar-casos` pasa
- [ ] Actualicé `docs/reglas-compilador.md` o `docs/validadores.md`
- [ ] No modifiqué archivos de `contrato/` ni de `tests/aceptacion/`
- [ ] Respeté `docs/convenciones-fisicas.md`
- [ ] No hay claves, datos personales ni texto copiado de libros

## Revisión cruzada de casos de oro
Casos escritos por: @
Casos revisados por (otra persona): @

## Cómo probarlo
<!-- Comando exacto y resultado esperado. -->

## Dudas o puntos a revisar
```

Plantilla de issue

Los criterios de aceptación se escriben como condiciones verificables, no como intenciones generales. Los issues llevan etiquetas de módulo y dificultad.

12. Convenciones físicas (`docs/convenciones-fisicas.md`)

Este documento es más importante de lo que parece. Si una persona asume eje positivo hacia abajo y otra hacia arriba, las dos tienen “tests que pasan” y el sistema integrado falla. Ahí se fijan: sentido de los ejes, signo del rozamiento, valor de g ($9,8 \text{ m/s}^2$), unidades internas (SI) y cómo se representan los ángulos (en la escena pueden venir en grados; dentro del código siempre en radianes). **Léanlo antes de escribir nada.**

Los ADR (`docs/decisiones/`) son páginas cortas con contexto, decisión y alternativas descartadas. Sirven para el informe final y para que nadie reabra decisiones cada semana.

13. Qué NO va en el repositorio

- **Claves y secretos:** nunca. Usen variables de entorno y un `.env.example` sin valores reales, más `.gitignore` y un escaneo de secretos en el CI.
- **Libros y PDFs con derechos de autor:** ni los originales ni su conversión con Marker.
- **Datos de alumnos** o logs del piloto.

14. Integración y versionado

Como el motor se desarrolla aparte y se integra después, el riesgo se mueve a la integración: si cada lado trabaja aislado durante semanas, los problemas aparecen todos juntos al final. Estas medidas apuntan a evitarlo:

1. **Una única puerta de entrada:** `resolver(escena) -> Resultado` y su adaptador `resultado_a_grafico()` (ver 7.5). El equipo central integra contra esas dos funciones y nada más.
2. **Contrato versionado y congelado:** el esquema se publica como `esquema_escena.schema.json` con número de versión (1.0.0). Si hay que cambiarlo, se sube la versión y se avisa. Los cambios al contrato pasan por el equipo central, nunca por los colaboradores.
3. **Versionado semántico con tags:** cada entrega es un tag (`v0.1.0`, `v0.2.0`). El backend fija una versión concreta y la actualiza a propósito (`pip install git+https://...@v0.3.0`).
4. **Integraciones tempranas y frecuentes:** un hito cada 2 semanas aproximadamente. En cada uno, el equipo central instala la versión del momento y corre un test de integración chico. Mientras tanto, el backend avanza con un motor “de mentira” que devuelve respuestas fijas con el formato del contrato.
5. **Mismas versiones de dependencias:** SymPy, SciPy, Pint y Pydantic fijados con lock file, y la misma versión de Python que tendrá la imagen Docker del servidor.
6. **Test de aceptación del equipo central:** 5 a 10 casos de oro propios en `tests/aceptacion/`, que solo el equipo central edita. Si el motor los pasa al integrarlo, la entrega es aceptada. Así nadie puede “aprobarse a sí mismo” con casos que escribió.

15. Qué entrega el equipo central antes de abrir el repo

- La firma de `resolver()`, el modelo `Resultado`, el adaptador `resultado_a_grafico()` al formato de 5.2 y el formato de los errores (`contrato/CONTRATO.md`).
- El `esquema_escena.schema.json` en versión 1.0.0.
- `convenciones-fisicas.md` completo (ejes, signos, g , unidades).
- Una regla completa de punta a punta como ejemplo (el *walking skeleton*, por ejemplo la normal en el plano inclinado) con su código, su caso de oro, su test unitario, su test de propiedad y su entrada en `docs/reglas-compilador.md`. **Es la mejor documentación posible: copien el patrón en vez de interpretarlo.**

- Los 5 a 10 casos de aceptación propios.
- El calendario de hitos de integración, para que sepan qué se espera en cada uno.

Orden de puesta en marcha:

| Paso | Qué | Quién |
|------|--|---|
| 1 | Esqueleto de carpetas, pyproject, Dockerfile, Makefile, CI | Equipo central (1 persona de infraestructura) |
| 2 | Esquema de la escena y contratos | Equipo central |
| 3 | Walking skeleton (una regla completa) | Equipo central |
| 4 | Docs: glosario, convenciones físicas, formato de caso de oro | Equipo central |
| 5 | Prueba del onboarding con un colaborador que no participó | Colaborador voluntario |
| 6 | Apertura a todos los colaboradores con issues ya creados | Equipo central |

El paso 5 es el más subestimado: si alguien nuevo no llega a verde en 10 minutos, es un fallo del repo y no de la persona.

16. Cómo se evalúa el trabajo

1. **Testeo aislado con pytest:** cada función o herramienta se prueba sola, contra los casos de referencia del equipo central y los casos de oro.
2. **Validadores cruzados:** un par de personas de otros grupos revisan los cálculos físicos antes de integrar cada pieza al sistema completo. La persona que escribe no es la única que valida su propio cálculo.
3. **Golden set:** un conjunto de ejercicios reales de la cátedra (con resolución oficial) contra los que se corre todo el sistema integrado, para medir qué porcentaje resuelve bien.

17. Dudas y coordinación

Como los grupos trabajan en paralelo sobre piezas que tienen que encajar, hay una reunión de sincronización periódica con el equipo central para resolver dudas sobre los contratos universales (sección 5) y evitar que cada grupo interprete las reglas compartidas de forma distinta. Si tienen una duda sobre si algo “cuenta como” una excepción al contrato, consulten antes de implementar a su criterio.

18. Lista de tareas pendientes de los equipos desarrolladores

Esta lista reúne todo lo que queda por hacer, agrupado por responsable y por pieza. Las secciones B, C y D corresponden al motor (Grupo C); la sección G recoge las tareas de la pieza ODE (Grupo B) que se desprenden de las reglas compartidas. Cada tarea se cierra con un criterio verificable (un test en verde, un caso de oro que pasa, un documento publicado), no con “implementar X”.

A. Antes de empezar (equipo central)

1. Definir en `contrato/CONTRATO.md` la firma de `resolver(escena) -> Resultado`, el modelo `Resultado`, el adaptador `resultado_a_grafico(resultado) -> dict` (formato de 5.2) y el formato de los errores.
2. Publicar `esquema_escena.schema.json` en versión 1.0.0.
3. Escribir `docs/convenciones-fisicas.md`: ejes, signo del rozamiento, $g = 9,8 \text{ m/s}^2$, unidades SI y ángulos en radianes dentro del código.
4. Armar el esqueleto del repo `fisica-motor`: `pyproject`, `uv.lock`, `Dockerfile`, `Makefile`, `CI`, `CODEOWNERS` y plantillas de PR e issue.
5. Implementar un “walking skeleton”: una regla completa de punta a punta (por ejemplo, la normal en el plano inclinado) con su caso de oro, su test unitario, su test de propiedad y su entrada en `docs/reglas-compiler.md`.
6. Escribir `glosario.md`, el formato de caso de oro y `herramientas/validar_casos.py`.
7. Escribir 5 a 10 casos de aceptación propios en `tests/aceptacion/`.
8. Fijar el calendario de hitos de integración, uno cada 2 semanas aproximadamente.
9. Probar el onboarding con un voluntario que no haya participado: debe llegar a verde en 10 minutos.
10. Convocar a los ayudantes de cátedra y a los tres grupos para que **cada grupo defina su propio alcance de temas** (secciones 5.9 y 7.6). El equipo central no decide el alcance: verifica que los alcances no se solapen ni dejen huecos, registra los acuerdos y coordina cualquier cambio posterior.

B. Compiler

1. Definir, junto con los ayudantes de cátedra, el alcance de temas del compiler (qué situaciones entran y cuáles no) y documentarlo antes de implementar reglas. Una vez acordado, no ampliarlo sin avisar al resto de los grupos ni al equipo central.
2. Implementar los modelos Pydantic de la escena (`src/motor/esquema/`): cuerpo, superficie, contacto, cuerda, polea, resorte, condiciones iniciales e incógnitas.
3. Armar el diagrama de cuerpo libre automático (peso, normal, rozamiento, tensión, fuerza elástica), una regla por archivo.
4. Elegir los ejes de proyección y proyectar las fuerzas.
5. Escribir la segunda ley de Newton ($\Sigma F = m \cdot a$) por cuerpo y por eje.
6. Agregar las restricciones: cuerda inextensible (válida solo con la cuerda tensa), polea ideal y resorte.
7. Manejar el rozamiento por hipótesis (desliza o no desliza), verificando su consistencia (por ejemplo, que el rozamiento estático no supere su límite) y probando la otra hipótesis si no cierra.
8. Manejar la hipótesis de cuerda (tensa o floja): poder replantear sin la restricción de cuerda inextensible ($T = 0$) cuando el motor lo pida tras una tensión negativa.
9. Convertir unidades de entrada (por ejemplo, grados a radianes) y validar la entrada con `ParametroInvalidoError`.
10. Documentar cada regla y su estado en `docs/reglas-compiler.md`.

C. Solver

1. Hacer el envoltorio de `sympy.solve()` para sistemas algebraicos y de `sympy.dsolve()` para ecuaciones diferenciales con solución exacta (`src/motor/solver/`).
2. Detectar cuando no hay solución y lanzar `CalculoNoConvergeError`. Incluye el caso en que `dsolve` no lanza excepción pero devuelve una integral sin evaluar, una solución implícita o una lista de ramas.

3. Trabajar solo con ecuaciones simbólicas construidas por el compilador a partir de la escena estructurada: el solver nunca recibe texto libre ni usa `eval()` ni `exec()`. (El parseo seguro de fórmulas en texto es tarea de la pieza ODE, ver sección G.)
4. Asegurar que todo sea función pura y síncrona, sin estado global.
5. Fijar las versiones de SymPy, SciPy, Pint y Pydantic en el lock file.

D. Validador

1. Implementar `validar_tension_no_negativa`.
2. Implementar `validar_suma_fuerzas_igual_ma`, con tolerancia 0,05.
3. Implementar `validar_normal_perpendicular`, con tolerancia 0,01.
4. Declarar en `docs/validadores.md`, para cada validador, su severidad, su regla física y la acción ante un fallo (qué causa física cuestiona o si corresponde error directo).
5. Ante una tensión negativa, replantear con la cuerda floja ($T = 0$, sin restricción de cuerda inextensible), volver a resolver y validar; si sigue fallando, devolver el error. No cambiar la hipótesis de rozamiento para este caso.
6. Ante un fallo de los otros validadores (suma de fuerzas, normal perpendicular), no reintentar con otra hipótesis: no graficar y devolver el error para que el asistente lo explique en texto.

E. Tests y casos de oro (todos)

1. Escribir primero el caso de oro y el test, y después el código.
2. Cubrir con tests unitarios, de propiedades (Hypothesis), adversos (entradas que deben fallar de forma controlada) y de regresión (un caso por cada bug).
3. Cargar casos de oro en `dinamica/` y `cinematica/`, parafraseando los enunciados.
4. Hacer revisión cruzada: `revisado_por` distinto de `autor`, y pasar el caso a `revisados/`.
5. Usar PR chicos, de una regla o un validador cada uno, con CI en verde y una aprobación.

F. Integración (equipo central)

1. Publicar tags de versión (`v0.1.0`, `v0.2.0`, ...) y fijar una versión en el backend.
2. Armar un motor “de mentira” con respuestas fijas para que el backend avance en paralelo.
3. Correr un test de integración chico en cada hito, que verifique que `resultado_a_grafico(resolver(escena))` cumple el formato de la sección 5.2.
4. Pasar el golden set de ejercicios reales de la cátedra sobre el sistema integrado y medir el porcentaje resuelto.

G. Pieza ODE (Grupo B): tareas derivadas de las reglas compartidas

1. Parsear `fuerza_expresion` solo con `sympy` o `asteval`, nunca con `eval()` ni `exec()`.
2. Validar antes de integrar que la expresión use solo `x`, `v`, `t` y números; si no, lanzar `ParametroInvalidoError`.
3. Implementar el criterio de solución exacta de la sección 6 (`solucion_exacta_o_none`): no asumir que `dsolve` lanza `NotImplementedError`, y descartar integrales sin evaluar, soluciones implícitas y listas de ramas.
4. Integrar con `solve_ivp` (RK45 por defecto) y devolver `CalculoNoConvergeError` si no tiene éxito.